

DOCUMENTO DE ARQUITETURA DE SOFTWARE

Projeto: Livraria Online

Versão	1.0
Data	Abril de 2025
Disciplina	Engenharia de Software
Status	Em desenvolvimento

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
Abril / 2025	1.0	Criação inicial do documento de arquitetura	Equipe do Projeto

Sumário

- 1. Introdução
 - 1.1 Finalidade
 - 1.2 Escopo
 - 1.3 Definições e Acrônimos
- 2. Descrição Geral da Arquitetura
- 3. Estilo Arquitetural Adotado
- 4. Requisitos e Restrições Arquiteturais
- 5. Principais Componentes e Responsabilidades
- 6. Diagrama de Arquitetura
- 7. Tecnologias Previstas
- 8. Visão Lógica – Camadas e Pacotes
- 9. Visão de Implantação
- 10. Justificativa das Decisões Arquiteturais
- 11. Qualidade e Requisitos Não-Funcionais

1. Introdução

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema Livraria Online, descrevendo as principais decisões de projeto tomadas pela equipe de desenvolvimento, os critérios considerados durante a tomada dessas decisões e a estrutura técnica que suporta o funcionamento da plataforma.

O objetivo é capturar e comunicar as decisões arquiteturais significativas, permitindo o entendimento do sistema por diferentes stakeholders — equipe técnica, clientes e demais envolvidos no projeto.

1.2 Escopo

Este Documento de Arquitetura de Software aplica-se ao sistema Livraria Online, uma plataforma digital de publicação e venda de obras literárias voltada a autores independentes e leitores de baixa a média renda. O sistema será desenvolvido pela equipe da disciplina de Engenharia de Software e envolve os seguintes módulos principais:

- Módulo de catálogo e busca de obras
- Módulo de publicação e gerenciamento de obras (painel do autor)
- Módulo de compra/acesso e checkout
- Módulo de revisão assistida por IA (correção gramatical)
- Módulo de avaliação e certificação humana de qualidade
- Módulo de gestão de usuários e autenticação

1.3 Definições, Acrônimos e Abreviações

Termo	Definição
API	Application Programming Interface – interface de comunicação entre sistemas
REST	Representational State Transfer – estilo arquitetural para APIs web
ORM	Object-Relational Mapping – mapeamento objeto-relacional para banco de dados
CI/CD	Continuous Integration / Continuous Deployment – integração e entrega contínuas
IA	Inteligência Artificial – tecnologia usada para correção gramatical e análise de texto
SPA	Single Page Application – aplicação web de página única (ex: React)
JWT	JSON Web Token – padrão para autenticação e troca segura de informações
Docker	Plataforma de containerização para padronizar ambientes de desenvolvimento e produção
CDN	Content Delivery Network – rede de entrega de conteúdo para arquivos estáticos

2. Descrição Geral da Arquitetura

O sistema Livraria Online é uma plataforma web full-stack projetada para democratizar o acesso à leitura e à publicação independente de obras literárias. A plataforma atende dois perfis principais de usuário: autores independentes, que publicam suas obras e acompanham métricas de venda/acesso, e leitores, que navegam pelo catálogo, adquirem e acessam os livros.

A arquitetura foi concebida seguindo o estilo em camadas (Layered Architecture) com separação clara entre frontend, backend, serviços externos e persistência de dados. A comunicação entre o frontend e o backend ocorre por meio de uma API RESTful. Um serviço de IA externo é integrado para fornecer suporte a correção gramatical e análise textual das obras submetidas, enquanto revisores humanos realizam a aprovação final para certificação de qualidade.

A plataforma opera na nuvem, utilizando containers Docker para garantir portabilidade e padronização dos ambientes de desenvolvimento e produção. O deploy é realizado em plataformas modernas como Vercel (frontend) e Render ou Railway (backend), que oferecem pipelines de CI/CD automatizados.

3. Estilo Arquitetural Adotado

O estilo arquitetural principal adotado é a Arquitetura em Camadas (N-Tier / Layered Architecture), combinada com o padrão Cliente-Servidor. Essa abordagem organiza o sistema em camadas com responsabilidades bem definidas e isoladas, promovendo manutenibilidade, testabilidade e escalabilidade.

Camada	Padrão / Estilo	Justificativa
Apresentação	SPA (Single Page Application)	Proporciona experiência de usuário fluida e responsiva sem recarregamento de página
Comunicação	REST API (HTTP/JSON)	Padrão consolidado, stateless, fácil de consumir por diferentes clientes
Aplicação/Negócio	MVC / Service Layer	Separação entre controllers, regras de negócio e acesso a dados
Persistência	ORM (Object-Relational Mapping)	Abstração do banco de dados, facilitando manutenção e migração
Infraestrutura	Containers + Cloud PaaS	Portabilidade, padronização de ambientes e deploy simplificado

4. Requisitos e Restrições Arquiteturais

Requisito	Solução Adotada
Linguagem de Backend	Python (Django REST Framework ou FastAPI)
Linguagem de Frontend	JavaScript/TypeScript com React.js (Next.js)
Banco de Dados	PostgreSQL (produção) ou MySQL como alternativa
Segurança / Autenticação	JWT (JSON Web Token) para autenticação stateless; HTTPS obrigatório; hash de senhas com bcrypt
Persistência	ORM (Django ORM ou SQLAlchemy) com migrations versionadas
Integração com IA	Chamadas via API REST a serviço de IA (OpenAI ou similar) para correção gramatical
Internacionalização (i18n)	Sistema desenvolvido em Português Brasileiro (pt-BR); estrutura pronta para expansão
Escalabilidade	Containers Docker + deploy em plataformas de nuvem com suporte a auto-scaling
Disponibilidade	Deploy em provedores de nuvem com uptime garantido (Vercel, Render, Railway)
Controle de Versão	Git + GitHub/GitLab para versionamento de código e colaboração em equipe

5. Principais Componentes e Responsabilidades

5.1 Frontend (Camada de Apresentação)

Responsável pela interface gráfica do sistema, com a qual os usuários interagem diretamente. Desenvolvido com React.js (preferencialmente com Next.js para suporte a SSR e otimizações de SEO).

- Página inicial com catálogo de obras e sistema de busca/filtragem
- Páginas de detalhes de obras (sinopse, autor, avaliações, preço)
- Painel do autor: publicação de obras, gerenciamento de manuscritos, status de revisão
- Fluxo de compra/acesso: carrinho, checkout e biblioteca pessoal do leitor
- Gerenciamento de conta e perfil de usuário
- Painel administrativo (para revisores humanos aprovarem obras)

5.2 Backend / API (Camada de Aplicação)

Responsável por toda a lógica de negócio da plataforma. Expõe uma API RESTful consumida pelo frontend e por integrações externas.

- Autenticação e autorização de usuários (JWT, controle de papéis: leitor, autor, revisor, admin)
- CRUD de obras: criação, edição, exclusão, consulta
- Orquestração do fluxo de revisão: submissão > IA > revisor humano > publicação
- Processamento de pedidos e controle de acesso a obras adquiridas
- Integração com serviço de IA para análise e correção de manuscritos
- Endpoints de busca e listagem com filtros (gênero, preço, avaliação)

5.3 Serviço de IA (Camada de Serviços Externos)

Módulo de integração com API de inteligência artificial (OpenAI ou similar) para suporte ao processo editorial.

- Correção gramatical e ortográfica automática de manuscritos enviados
- Sugestões de melhorias textuais para os autores
- Análise de legibilidade e estrutura do texto
- Geração de relatório de revisão automática para o revisor humano

5.4 Banco de Dados (Camada de Persistência)

Responsável pelo armazenamento e recuperação de todos os dados do sistema.

- Tabelas principais: Usuários, Obras, Pedidos, Avaliações, Revisões, Categorias
- Controle de acesso por obra (relação entre usuário e obras adquiridas)
- Armazenamento de metadados de obras (título, sinopse, autor, preço, status)
- Histórico de revisões e status do fluxo editorial

5.5 Infraestrutura (Camada de Implantação)

- Docker para containerização de todos os serviços (backend, banco de dados)
- Frontend hospedado na Vercel com CDN global e deploy automático

- Backend hospedado no Render ou Railway com suporte a containers
- Banco de dados PostgreSQL gerenciado na nuvem (Railway ou Render Postgres)
- Armazenamento de arquivos (PDFs/EPUBs das obras) em serviço de object storage (ex: AWS S3 ou Cloudflare R2)

6. Diagrama de Arquitetura

O diagrama a seguir representa a estrutura em camadas do sistema Livraria Online, ilustrando os principais blocos arquiteturais, suas tecnologias e o fluxo de comunicação entre eles.

Camada	Tecnologia	Responsabilidade
CAMADA DE APRESENTAÇÃO (Frontend)	React.js / Next.js	Interface do usuário, páginas de catálogo, checkout, painel do autor
CAMADA DE APLICAÇÃO (Backend)	Python / Django REST / FastAPI	Regras de negócio, autenticação, orquestração de serviços, API RESTful
CAMADA DE SERVIÇOS EXTERNOS	OpenAI API / Serviço de IA	Correção gramatical, sugestões de texto, análise de qualidade
CAMADA DE PERSISTÊNCIA	PostgreSQL / MySQL	Dados de usuários, obras, pedidos, avaliações
CAMADA DE INFRAESTRUTURA	Docker + Vercel / Render / Railway	Containerização, deploy, escalabilidade, CI/CD

6.1 Fluxo Principal de Comunicação

O fluxo padrão de uma requisição no sistema segue o caminho:

- 1. O usuário interage com a interface no navegador (React/Next.js)
- 2. O frontend realiza chamadas HTTP/REST para a API do backend (Python/Django ou FastAPI)
- 3. O backend processa a requisição, aplica regras de negócio e consulta o banco de dados (PostgreSQL)
- 4. Quando necessário, o backend aciona o serviço de IA (via API REST externa) para análise de manuscritos
- 5. O resultado é retornado em formato JSON ao frontend, que atualiza a interface

6.2 Diagrama de Fluxo do Processo Editorial

Etapa	Responsável	Ação
1. Submissão	Autor	Upload do manuscrito pela plataforma
2. Análise por IA	Serviço de IA (automático)	Correção gramatical, análise de legibilidade e geração de relatório
3. Revisão Humana	Revisor / Editor	Leitura da obra e avaliação de qualidade com base no relatório da IA
4. Aprovação / Rejeição	Revisor	Aprovação com selo de qualidade ou devolução ao autor com feedback
5. Publicação	Sistema (automático)	Obra disponibilizada no catálogo público com selo de certificação

7. Tecnologias Previstas

Categoria	Tecnologia	Versão / Observação
Linguagem Backend	Python	3.11+
Framework Backend	Django REST Framework ou FastAPI	Django 4.x / FastAPI 0.100+
Linguagem Frontend	JavaScript / TypeScript	TypeScript 5.x (recomendado)
Framework Frontend	React.js com Next.js	Next.js 14+ (App Router)
Banco de Dados	PostgreSQL	15+ (principal); MySQL como alternativa
ORM	Django ORM ou SQLAlchemy	Nativo do framework escolhido
Autenticação	JWT (JSON Web Token)	Bibliotecas: djangorestframework-simplejwt ou PyJWT
Serviço de IA	OpenAI API ou similar	Modelo GPT-4o ou GPT-3.5-turbo para correção textual
Containerização	Docker + Docker Compose	Docker 24+
Hospedagem Frontend	Vercel	Deploy automático via Git
Hospedagem Backend	Render ou Railway	Suporte a containers e banco de dados gerenciado
Armazenamento de Arquivos	AWS S3 ou Cloudflare R2	Para PDFs e EPUBs das obras
Controle de Versão	Git + GitHub	GitFlow ou trunk-based development
CSS / UI	Tailwind CSS ou Material UI	Para estilização rápida e consistente
Testes	pytest (backend) + Jest (frontend)	Testes unitários e de integração

8. Visão Lógica – Camadas e Pacotes

8.1 Estrutura de Pacotes do Backend (Python/Django)

A organização do projeto backend segue a convenção de apps do Django, cada um responsável por um domínio específico do negócio:

App / Módulo	Responsabilidade Principal
users	Gerenciamento de usuários, autenticação, perfis (autor, leitor, revisor, admin)
books	CRUD de obras, categorias, metadados, armazenamento de arquivos
editorial	Fluxo de submissão, revisão por IA, aprovação humana, status editorial
orders	Processamento de pedidos, histórico de compras, controle de acesso a obras
reviews	Avaliações e comentários de leitores sobre as obras
ai_service	Integração com API de IA: envio de manuscritos e processamento de resposta
core / config	Configurações globais, middlewares, rotas principais, utilitários comuns

8.2 Estrutura de Páginas/Componentes do Frontend (Next.js)

Módulo / Página	Responsabilidade Principal
/ (Home)	Catálogo de obras em destaque, busca e filtros
/books/[id]	Página de detalhes de uma obra
/auth	Login, cadastro e recuperação de senha
/dashboard/author	Painel do autor: gerenciar obras, status de revisão, métricas
/dashboard/library	Biblioteca do leitor: obras adquiridas e acesso ao conteúdo
/admin	Painel do revisor/admin: fila de revisão, aprovação e certificação
/checkout	Fluxo de compra e pagamento
/components	Componentes reutilizáveis: cards, modais, formulários, layouts
/services / api	Camada de comunicação com a API backend (fetch/axios)

9. Visão de Implantação

A seguir está descrita a topologia de implantação do sistema em produção, baseada em serviços de nuvem e containers.

Nó / Serviço	Tecnologia	Função
Servidor Frontend	Vercel (CDN global)	Hospeda e serve a aplicação React/Next.js; deploy automático via push no Git
Servidor Backend	Render ou Railway	Executa o container Docker com a API Python; gerencia variáveis de ambiente e secrets
Banco de Dados	PostgreSQL gerenciado (Railway/Render)	Persistência de todos os dados da aplicação com backups automáticos
Object Storage	AWS S3 ou Cloudflare R2	Armazenamento dos arquivos das obras (PDF, EPUB) com acesso controlado por assinatura temporária
Serviço de IA	OpenAI API (externo)	Processamento de manuscritos via chamadas REST autenticadas com chave de API
Repositório de Código	GitHub	Controle de versão, pull requests e gatilho de pipelines de CI/CD

10. Justificativa das Decisões Arquiteturais

10.1 Python como linguagem de Backend

Python foi escolhido por sua sintaxe clara e curva de aprendizado suave, o que é estratégico para um projeto acadêmico desenvolvido por equipe com diferentes níveis de experiência. Além disso, Python possui um ecossistema maduro para desenvolvimento web (Django, FastAPI), integração com IA (OpenAI SDK, LangChain, Transformers) e ferramentas de teste. Essa sinergia entre produtividade e integração com IA foi fator decisivo.

10.2 Django REST Framework vs FastAPI

Ambos são excelentes opções. Django REST Framework foi considerado pela sua convenção sobre configuração, ORM nativo e painel administrativo embutido (útil para o módulo de revisão). FastAPI foi considerado por sua performance superior e tipagem com Pydantic. A decisão final depende da preferência da equipe, porém ambos foram listados como opções viáveis na documentação.

10.3 React.js com Next.js no Frontend

React.js é a biblioteca de UI mais difundida no mercado, com comunidade e ecossistema gigantes. O Next.js acrescenta funcionalidades críticas como Server-Side Rendering (SSR), otimização de imagens, roteamento baseado em arquivos e suporte nativo a APIs. Para uma plataforma de e-commerce literário, SEO é essencial (as obras precisam ser encontradas via mecanismos de busca), o que torna o SSR uma vantagem competitiva clara.

10.4 PostgreSQL como banco de dados

PostgreSQL é um banco relacional robusto, open-source e amplamente suportado por todas as plataformas de nuvem consideradas. Sua confiabilidade, suporte a tipos avançados (JSON, arrays) e ferramentas visuais como pgAdmin facilitam o desenvolvimento. O modelo de dados do projeto (usuários, obras, pedidos, revisões) é claramente relacional, favorecendo a escolha de um banco SQL.

10.5 Docker para containerização

Docker é indispensável para garantir que o ambiente de desenvolvimento seja idêntico ao de produção, eliminando problemas do tipo 'funciona na minha máquina'. O uso de Docker Compose facilita a orquestração local dos serviços (backend + banco de dados), enquanto em produção os containers são gerenciados pelas plataformas de nuvem escolhidas.

10.6 Integração de IA no processo editorial

A inclusão de um serviço de IA para revisão gramatical é um diferencial da plataforma e está diretamente alinhada à proposta de negócio. A IA não substitui o revisor humano — ela agiliza o processo editorial, gerando um relatório preliminar que o revisor usa como base para a avaliação de qualidade. Isso garante escalabilidade do processo de revisão mesmo com aumento no volume de submissões.

10.7 Arquitetura em Camadas

O estilo arquitetural em camadas foi adotado por ser bem documentado, amplamente compreendido pela equipe e alinhado às boas práticas de Engenharia de Software. A separação clara entre apresentação, aplicação e persistência facilita a manutenção, os testes unitários por camada e a evolução independente de cada parte do sistema.

11. Qualidade e Requisitos Não-Funcionais

Atributo de Qualidade	Descrição	Solução / Estratégia
Desempenho	Respostas rápidas para buscas e carregamento de páginas	Next.js com SSR/SSG para páginas críticas; cache de consultas frequentes; CDN para assets estáticos
Segurança	Proteção de dados de usuários e obras	HTTPS obrigatório; autenticação JWT; hash de senhas com bcrypt; validação de inputs; controle de acesso por papel
Disponibilidade	Sistema acessível continuamente	Deploy em provedores com SLA; containers com restart automático; monitoramento básico
Escalabilidade	Suportar crescimento de usuários e obras	Arquitetura stateless no backend; containers escaláveis horizontalmente; banco de dados gerenciado na nuvem
Manutenibilidade	Código legível e fácil de evoluir	Separação em camadas; padrões REST; documentação de API (OpenAPI/Swagger); testes automatizados
Usabilidade	Interface intuitiva para autores e leitores	UI com componentes padronizados (Tailwind/MUI); design responsivo; fluxos simplificados de compra e publicação
Confiabilidade	Dados íntegros e fluxos sem falhas	Transações no banco de dados; tratamento de erros na API; logs de operações críticas

Referências

- DATASUS – Template de Documento de Arquitetura de Software (Ministério da Saúde / DATASUS)
- KRUCHTEN, Philippe. The '4+1' View Model of Software Architecture. IEEE Software, 1995.
- Django REST Framework – <https://www.django-rest-framework.org/>
- FastAPI Documentation – <https://fastapi.tiangolo.com/>
- Next.js Documentation – <https://nextjs.org/docs>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Docker Documentation – <https://docs.docker.com/>
- OpenAI API Documentation – <https://platform.openai.com/docs/>